

Содержание

1	Воспоминания о ФП	3
1.1	Термы и редукция	3
1.2	Типы	3
1.3	Функции в Haskell	6
1.4	Данные в Haskell	7
1.5	Классы типов в Haskell	9
1.6	Монады в Haskell	10
2	Параметрический полиморфизм	13
2.1	Параметрический полиморфизм в языке	13
2.1.1	First-class polymorphism	17
2.1.2	Higher-order/kinded polymorphism	19
2.1.3	Обобщённые алгебраические типы данных (GADTs)	20
2.1.4	Структуры на уровне типов, data promotion	21
2.1.5	Факультативно: эмуляция типовых абстракций и аппликаций (Proxy)	25
2.2	Реализация параметрического полиморфизма	25
2.2.1	Мономорфизация	26
2.2.2	Стирание типа	27
2.2.3	Гибридный подход	28
2.2.4	Использование виртуальной таблицы свойств типов	28
2.2.5	Сравнение подходов	29
2.3	Полиморфизм по конвенции вызова	30
2.3.1	Разновидности runtime представлений в Haskell	30
2.3.2	Классификация значений по runtime представлению	31
2.3.3	Representation polymorphism	32
3	Специальный (ad-hoc) полиморфизм	34
3.1	Классы типов в языке	35
3.1.1	Словари	35
3.1.2	Неявные аргументы	37
3.1.3	Вывод инстансов	38
3.1.4	Построение типа по значению	40

¹Автор Андрей Стоян (andrey.stoyan.csam@gmail.com).

²Спасибо Илье Колегову за первое внимательное прочтение и кучу комментариев. Спасибо моим студентам, которые своим интересом к предмету сделали эту работу возможной.

³Версия от 30 сентября 2026 г., 21:35.

Введение

Многие сложные концепции в дизайне языков и программ могут быть поняты как частные случаи некоторых простых фундаментальных принципов, которые, как правило, считаются общеизвестным фольклором, не требующим дополнительных пояснений. Однако, сложность в том, что эти знания рассеяны по книгам, статьям и “культовым” блог-постам, и требуется довольно много времени и сил для восстановления целостной картины.

Цель данного курса — собрать в одном месте такие фольклорные знания и организовать их в некоторую систему. Курс будет явным образом опираться на классические работы, исследующие принципы построения языков, и помогать в их изучении. Просмотр упоминаемых статей является важной частью самостоятельной работы в рамках курса.

Под функциональным программированием, вынесенным даже в заголовок курса, понимается трепетное отношение к понятию эффекта, которое в ФП, в отличие от других школ мысли, не считается аксиоматической данностью, но предметом для изучения, сознательного конструирования и аккуратного обращения. Этот подход оказывается очень полезным для изучения языков, построения выразительных языковых конструкций, а также является основой для продуктивного стиля программирования. Кроме того, функциональные языки сравнительно просты, в результате чего новые идеи и подходы нередко зарождаются в них и распространяются далее.

В качестве основного языка курса выбран Haskell, так как он, с одной стороны, воплощает в себе многие концепции, часто доведённые до некоторого логического завершения, и достаточно выразителен для кодирования других. С другой стороны, всё ещё является прикладным промышленным языком программирования.

В связи с широтой контекста, данный курс не всегда является глубоким. Так, детали реализации в GHC или теоретико-категорные основания вещей могут даваться в общем виде и без конкретики. В то же время, в плоскости языкового дизайна через оптику функционального программирования курс пытается быть максимально подробным.

Таким образом, данный курс может быть полезен тем, кто интересуется дизайном языков и красивыми обобщениями программистских концепций, хочет улучшить свои навыки проектирования API или планирует вести практическую деятельность на функциональных языках.

Пререквизитом к прохождению курса является знание основ функционального программирования: алгебраических типов данных, паттерн-матчинга, свёрток, параметрического полиморфизма, классов типов, базовых монад. Дополнительно будет полезным умение читать типовые дробы, знакомство с полиморфным λ -исчислением и кодированием Чёрча.

1 Воспоминания о ФП

В этом разделе мы вспомним основные концепции функционального программирования и языка Haskell.

1.1 Термы и редукция

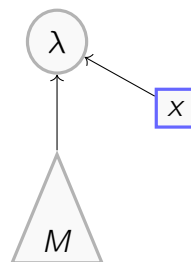
В ФП программы представляют собой выражения. Выполнение программ — редукция таких выражений до более “простых”. Выражения можно представлять как в виде линейной записи символов, так и в виде дерева, для понимания которого не требуется знания вспомогательных правил ассоциативности и проч.

Простейший функциональный язык — λ -исчисление. Выражения в нём называются λ -термами, которые состоят из вершин трёх видов (V — множество валидных идентификаторов, Λ — множество λ -термов):

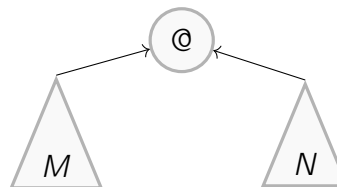
Переменные $x \in \Lambda$, если $x \in V$



Абстракция $(\lambda x. M) \in \Lambda$, если $x \in V, M \in \Lambda$



Аппликация $(M N) \in \Lambda$, если $M \in \Lambda, N \in \Lambda$



В произвольном выражении можно заменить некоторый его фрагмент на формальный параметр, который должен быть задекларирован выше по дереву с помощью специальной вершины λ . Вместо формального параметра можно в дальнейшем подставлять различные конкретные параметры с помощью вершины-аппликации $@$, то есть переиспользовать это выражение для различных целей (например, см. рис. 1). Редукция как раз определяется как следующее правило переписывания: ищется применение λ -функции к аргументу и в её тело осуществляется подстановка аргумента во все свободные вхождения переменной, связанной лямбдой (рис. 2).

1.2 Типы

Программное обеспечение — это сложно. Поэтому постоянно и неизбежно в программах возникают ошибки. Их можно искать, в том числе, статически, то есть без запуска программы. Одним из видов статического анализа является анализ типов.

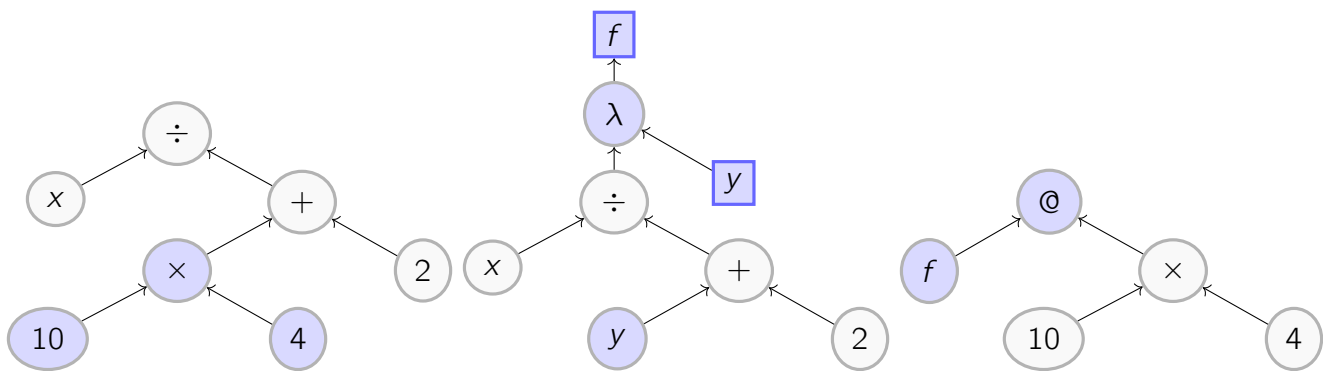


Рис. 1: Выражение с помощью λ вершины преобразуется в функцию одного аргумента.

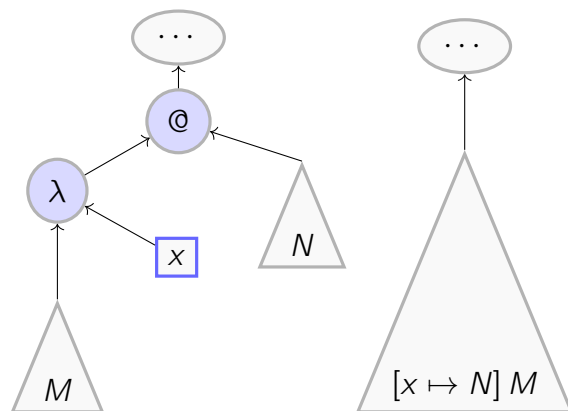


Рис. 2: Редукция переписывает дерево путём подстановки конкретного аргумента вместо формального параметра.

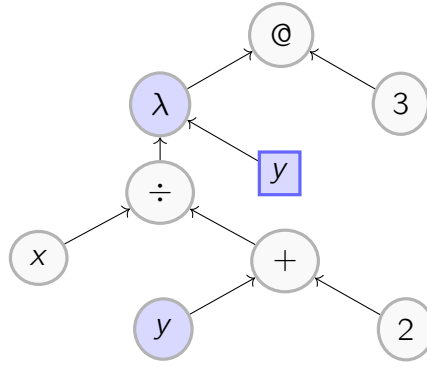


Рис. 3: Дерево соответствующее выражению $(\lambda y. x \div (y + 2)) 3$.

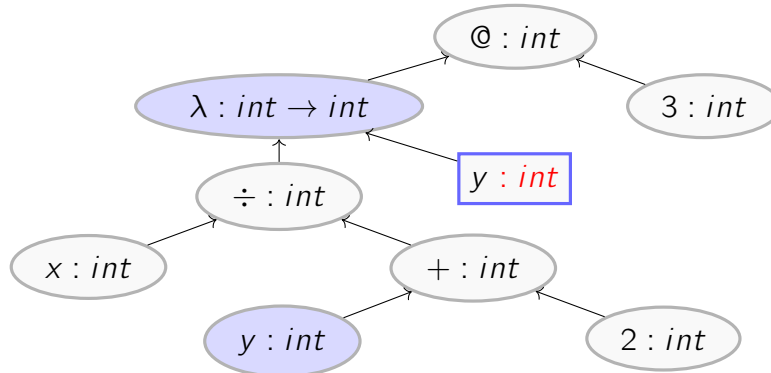


Рис. 4: Дерево выражения $(\lambda y. x \div (y + 2)) 3$ после присписывания типовых меток.

Идея анализа типов состоит в том, что каждой вершине дерева программы мы пытаемся приписать некоторую синтаксическую метку по определённым правилам. Если таким образом каждой вершине можно приписать метку, то мы считаем, что программа проходит проверку типов, и она “хорошая”. Например, на рисунке 3 представлено выражение, а на рисунке 4 каждой вершине приписаны метки в согласии с некоторой системой типов.

Система типов определяет синтаксис типовых меток и правила, по которым их можно приписывать. Синтаксис обычно описывается в классических нотациях а ля BNF, а правила в виде типовых дробей. Например, так выглядят дроби для просто-типизированного λ -исчисления:

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

Типовые метки имеют чисто-синтаксическую природу, однако их можно проинтерпретировать. Самая популярная интерпретация — воспринимать типовую метку как множество. Так, метке $int \rightarrow int$ можно поставить в соответствие множество функций между множествами целых чисел $\mathbb{Z} \rightarrow \mathbb{Z}$.

1.3 Функции в Haskell

В своей основе Haskell представляет собой расширенное типизированное λ -исчисление, дополненное примитивными типами, возможностью декларировать новые имена, структурами данных и классами типов.

Примеры λ -абстракций в REPL окружении GHCi:

```
1 ghci> (\x -> x + 1) 4
2 5
```

Можно узнать тип функции в интерпретаторе (в реальности числа полиморфные, но об этом далее):

```
1 ghci> :t \x -> x + 1
2 \x -> x + 1 :: Int -> Int
```

Функциям можно давать имена. Именам можно приписывать типы, это рекомендуется делать явно для деклараций на верхнем уровне файлов исходного кода:

```
1 f :: Int -> Int
2 f x = x + 1
```

Если имя типа начинается с маленькой буквы, то это не конкретный тип, а типовая переменная, способная принимать различные значения в зависимости от места вызова. Такая возможность называется **параметрическим полиморфизмом**. Так, функция, которая просто возвращает свой аргумент, никак не ограничивает тип аргумента. Но в то же время тип результата должен совпадать с типом аргумента:

```
1 id :: a -> a
2 id x = x

3 ghci> :t id 'x'
4 id 'x' :: Char
```

Функции могут принимать другие функции в качестве аргументов (такие функции называются **функциями высших порядков (higher-order functions)**). Имя функции может состоять из специальных символов, тогда она считается оператором и может применяться к своим операндам в инфиксном стиле:

```
1 ($) :: (a -> b) -> a -> b
2 f $ x = f x
```

Пример рекурсивной функции, использующей охранные выражения для отличия базового случая рекурсии:

```

1 factorial :: Int -> Int
2 factorial n
3   | n < 1 = 1
4   | otherwise = n * factorial (n - 1)

```

Упражнение 1 Что выведет запрос `ghci> :t uncurry (flip const)`?

Упражнение 2 Что выведет запрос `ghci> :t first . first` при

```

1 first :: (a -> a') -> (a, b) -> (a', b)

```

Упражнение 3 Реализуйте факториал с помощью техники аккумулирующего параметра.

1.4 Данные в Haskell

В Haskell есть встроенная возможность объявлять новые типы данных на основании других типов, а также создавать их экземпляры.

Зададим тип данных, описывающий животных:

```

1 data Animal
2   = Cat String Int
3   | Dog String

```

Мы задали тип данных `Animal` и два способа создать значения этого типа: для кошек и собак. `Cat` и `Dog` — это **конструкторы данных**. Они представляют собой функции, реализация которых находится на стороне языка. Они выделяют память под экземпляры данного типа и позиционно размещают компоненты. Кошек мы описываем именем и оставшимся количеством жизней, а собак — только именем.

```

1 Cat :: String -> Int -> Animal
2 Dog :: String -> Animal

```

Чтобы воспользоваться информацией, сохранённой в структуре данных, требуется деконструировать её с помощью паттерн-матчинга. Мы сопоставляем значение типа с образцом. Если образец похож на то, как было сконструировано значение, то он выбирается среди других образцов и переменные, задекларированные в нём, начинают ссылаться на соответствующее позиционно содержимое структуры данных:

```

1 describeAnimal :: Animal -> String
2 describeAnimal animal = case animal of
3   Cat name nLives -> "This is cat " ++ name ++ " " ++ show nLives
4   Dog name -> "This is dog " ++ name

```

В Haskell есть специальный синтаксис для объявления полей с именованными метками.

```

1 data Penguin = Penguin { getName :: String, getAge :: Int }
2 penguin = Penguin { getName = "Andrey", getAge = 500 }

```

Haskell генерирует функции-аксессоры для доступа к полям значения:

```

1 ghci> :t getName
2 getName :: Penguin -> String

```

Часто функции в программировании частичные — при некоторых значениях аргументов они могут вернуть результат, а при некоторых — нет. Давайте моделировать это с помощью специального типа данных. Если есть вещественный результат, будем возвращать его. Если нет, будем возвращать специально выделенное константное значение этого типа.

```

1 data MaybeD = NothingD | JustD Double
2 safeSqrtD :: Double -> MaybeD
3 safeSqrtD x = if x < 0 then NothingD else JustD (calcSqrt x)

```

Можно заметить, что так нам придётся объявлять отдельный тип вроде `MaybeD` для каждого типа `D`. Поэтому Haskell позволяет абстрагироваться в типе, аналогично тому как можно абстрагироваться по значениям в терме.

```

1 data Maybe a = Nothing | Just a
2 safeSqrt :: Double -> Maybe Double
3 safeSqrt x = if x < 0 then Nothing else Just (calcSqrt x)

```

Заметьте, что сейчас `Maybe` — это не совсем тип, так как теперь нужно передать типовой параметр, чтобы получить конкретный тип. `Maybe` называют **типовым конструктором**.

Вместе с абстракцией на уровне типов появилась и аппликация типа к типу. А что если дать меньше параметров типовому конструктору, чем ожидается? А что если больше? Контроль за корректностью типовых аппликаций обеспечивает **система кайндов**⁴. Это простейшие “типы для типов”, то есть синтаксические метки, контролирующие корректность записанных программистом типов. Так, обычные типы имеют метку (кайнд) `Type`⁵. Типовые конструкторы имеют стрелочные кайнды. Например, `Maybe :: Type -> Type`. Аппликация типового конструктора к типу подходящего кайнда убирает одну стрелку:

```

1 ghci> :k Int
2 Int :: Type
3 ghci> :k Maybe
4 Maybe :: Type -> Type
5 ghci> :k Maybe Int
6 Maybe Int :: Type

```

⁴Иногда в русскоязычной литературе кайнды называют родами типов, но мы не будем так говорить.

⁵Кайнд `Type` экспортируется из модуля `Data.Kind`. В старых текстах и в выводе GHCi по умолчанию он обозначается звёздочкой: `Maybe :: * -> *`. Это одно и то же; чтобы GHCi печатал `Type`, включите `:set -XNoStarIsType`.

Кроме совершенно новых типов данных, в Haskell можно объявлять типовые синонимы. Это имена, которые можно использовать вместо других типов, если, например, запись оригинального типа слишком длинная для повсеместного написания:

```
1 type T a = VeryLongType Int (a -> AnotherLongType a)
```

Если тип данных содержит только один конструктор и только одно поле, то отсутствует необходимость в аллокации новой памяти, содержащей тег конструктора и набор ссылок на поля. В таком случае, в качестве значения такого типа можно всегда просто использовать значение оборачиваемого типа, оставляя новый тип присутствовать исключительно во время компиляции, снижая нагрузку во время исполнения. Для объявления таких типов-обёрток нужно воспользоваться ключевым словом `newtype` вместо `data`:

```
1 newtype CourseId = CourseId Int64
2 newtype ModuleId = ModuleId Int64
```

Упражнение 4 Определите кайнд конструктора типа

```
1 data Free f a = Pure a | Free (f (Free f a))
```

1.5 Классы типов в Haskell

Параметрический полиморфизм позволяет использовать один и тот же код для различных типов входных данных. Классы типов же позволяют одному идентификатору ссылаться на разные реализации для разных типов данных (что аналогично механизму перегрузки (overloading) в других языках). Классы типов, как говорят, являются механизмом **специального (ad-hoc) полиморфизма**. Так, мы можем задекларировать символ `==`, выбор реализации которого зависит от выбора типа аргументов `a`:

```
1 class Eq a where
2   (==) :: a -> a -> Bool
```

Для каждого типа можно объявить свою собственную реализацию `Eq`:

```
1 instance Eq CourseId where
2   CourseId x == CourseId y = x == y

3 instance Eq a => Eq [a] where
4   [] == [] = True
5   x:xs == y:ys = x == y && xs == ys
6   _ == _ = False
```

Теперь в зависимости от конкретного типа `a` в месте вызова, будет выбрана подходящая реализация для этого типа:

```

1 ghci> CourseId 1 == CourseId 2
2 False
3 ghci> [CourseId 1, CourseId 2] == [CourseId 1, CourseId 2]
4 True

```

Рассмотренные ранее параметрически-полиморфные функции ничего не могли делать со своими аргументами, кроме как возвращать их в качестве результата или передавать в другие полиморфные функции. Чтобы уметь делать что-то ещё, нужна какая-то дополнительная информация про тип, потому что иначе нет никакой гарантии, что над объектом данного типа можно делать все необходимые операции. Так, функция `suc n = n + 1` не будет работать для строчек, потому что для них, очевидно, не определена операция сложения. Поэтому некорректно будет приписать полиморфный тип `suc :: a -> a`.

Классы типов, в отличие от перегрузки, в том числе являются механизмом ограничения полиморфности функций. Мы можем явно задать, что функция требует не произвольный тип на вход, а произвольный тип, для которого определены обязательно нужные нам операции. Так, для типа `suc` достаточно ограничить тип условием наличия плюса для него (операция обозначаемая символом `+` объявлена в классе типов `Num`):

```

1 suc :: Num a => a -> a

```

Упражнение 5 Реализуйте функцию, проверяющую равенство всех элементов данного списка.

Упражнение 6 Реализуйте инстанс полугруппы для функций.

Упражнение 7 Реализуйте инстанс `Eq` для функций. Подсказка: для произвольных функций это невозможно; подумайте, какие ограничения нужно наложить на тип аргумента, чтобы равенство стало проверяемым.

1.6 Монады в Haskell

Класс типов `Functor` объявляется для конструкторов типов и позволяет заменить в некотором контейнере все элементы одного типа на все элементы другого, оставляя структуру контейнера неизменной.

```

1 class Functor (f :: Type -> Type) where
2   fmap :: (a -> b) -> f a -> f b

3 instance Functor [] where
4   fmap :: (a -> b) -> [a] -> [b]
5   fmap _ [] = []
6   fmap f (x:xs) = f x : fmap f xs

```

В Haskell любая функция просто вычисляет результат некоторого типа. Однако в программировании часто требуются функции, которые не только вычисляют результат, но и делают что-то ещё. Например, изменяют какое-то состояние или пишут в консоль. Иными словами, производят побочные эффекты. В любом случае в Haskell мы можем вернуть из функции только результат, поэтому такие побочные эффекты мы кодируем в качестве дополнительной структуры, оборачивающей чистый результат. Т.е. если функция без побочных эффектов возвращала какой-то тип a , то после добавления побочных эффектов в её реализацию, она будет возвращать некоторый тип *вычислений* $f\ a$.

- Если функция кидает ошибку, то $f = \text{Maybe}$.
- Если функция читает глобальное состояние типа e , то $f = e \rightarrow _$.
- Если функция читает глобальное состояние s и обновляет его, то $f = s \rightarrow (s, _)$.

Стандартная библиотека Haskell предоставляет несколько классов типов для работы со значениями вида $f\ a$. Они позволяют абстрагироваться от структуры f и работать со значениями a внутри, как будто нет никакой дополнительной структуры.

Первый такой класс типов позволяет писать выражения над вычислениями $f\ a$.

```
1 class Functor f => Applicative (f :: Type -> Type) where
2   pure :: a -> f a
3   liftA2 :: (a -> b -> c) -> f a -> f b -> f c

4 instance Applicative Maybe where
5   pure :: a -> Maybe a
6   pure = Just

7   liftA2 :: (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c
8   liftA2 _ Nothing _ = Nothing
9   liftA2 _ _ Nothing = Nothing
10  liftA2 f (Just x) (Just y) = Just (f x y)
```

Второй класс типов позволяет делать последовательную композицию вычислений в императивном стиле⁶⁷:

```
1 class Applicative m => Monad (m :: Type -> Type) where
2   (>=) :: m a -> (a -> m b) -> m b

3 newtype State s a = State { runState :: s -> (s, a) }
```

⁶В коде ниже лямбда передаётся последним аргументом без скобок: `State \s -> ...` вместо `State (\s -> ...)`. Так позволяет писать расширение `BlockArguments`, мы будем пользоваться им повсеместно.

⁷В библиотеке `mtl` компоненты пары идут в обратном порядке: $s \rightarrow (a, s)$. Мы в конспекте ставим состояние первым.

```

4 instance Monad (State s) where
5   (>>=) :: State s a -> (a -> State s b) -> State s b
6   m >>= k = State \s ->
7     let (s', x) = runState m s in
8     runState (k x) s'

```

Теперь если мы определим базовые операции работы с состоянием, мы сможем писать код в императивном стиле с побочными эффектами.

```

1 get :: State s s
2 get = State \s -> (s, s)

3 put :: s -> State s ()
4 put newS = State \oldS -> (newS, ())

5 example :: State Int Int
6 example =
7   get >>= \x ->
8   put 42 >>= \() ->
9   get >>= \y ->
10  pure (x + y)

11 ghci> runState example 1
12 (42, 43)

```

Для таких монадических цепочек существует специальный синтаксический сахар:

```

1 example :: State Int Int
2 example = do
3   x <- get
4   put 42
5   y <- get
6   pure (x + y)

```

Упражнение 8 Реализуйте `liftA3` через `liftA2`.

Упражнение 9 Реализуйте `>>=` через `join` и наоборот.

Упражнение 10 Прочитайте два числа с консоли, поделите одно на другое нацело и распечатайте результат; если остаток не нулевой, распечатайте его тоже.

2 Параметрический полиморфизм

Никакое нетривиальное семантическое свойство программ не может быть алгоритмически проверено⁸. Чтобы оставаться разрешимыми (в смысле проверки типов и/или вывода), многие системы типов жертвуют полнотой и, помимо некорректных программ, отвергают много корректных. В то же время системы типов также стараются предоставлять различные возможности, позволяющие протипизировать как можно больше корректных программ. Одна из них — параметрический полиморфизм.

Под **параметрическим полиморфизмом** мы будем подразумевать возможность кода единообразно работать с произвольными типами данных Strachey [2000], Cardelli and Wegner [1985], что позволяет во многих случаях избегать дублирования кода.

В этой главе мы рассмотрим, как описывают полиморфизм в самом простом виде — в типизированном λ -исчислении. Изучим различные формы параметрического полиморфизма и сопутствующие техники безопасного программирования. Проанализируем возможные способы эффективной реализации параметрического полиморфизма. И в завершение рассмотрим полиморфизм по рантайм-представлению, “полиморфизм по полиморфизму”.

2.1 Параметрический полиморфизм в языке

Начнём со знакомого примера из предыдущей главы:

```
1 id :: a -> a
2 id x = x
```

Без полиморфизма нам пришлось бы написать отдельные `idInt`, `idChar` и так далее: корректные, но совершенно одинаковые программы. Типовая переменная `a` избавляет от дублирования, но оставляет вопрос: *кто и когда выбирает конкретный тип вместо `a`?*

Тип выбирает вызывающий код, причём отдельно в каждом месте вызова. Иначе говоря, у `id` есть ещё один, скрытый аргумент — сам тип. В Haskell этот аргумент можно сделать явным с обеих сторон: и принять его в определении функции, и передать в месте вызова⁹:

```
1 id :: forall a . a -> a
2 id @a (x :: a) = x

3 ghci> :t id @Int
4 id @Int :: Int -> Int
5 ghci> id @Char 'x'
6 'x'
```

Чтобы понять, откуда берётся скрытый аргумент, посмотрим, как полиморфизм описывают в самом простом виде — в типизированном λ -исчислении. Вспомним правила просто

⁸https://en.wikipedia.org/wiki/Rice%27s_theorem

⁹Расширения `TypeAbstractions` и `TypeApplications`; явный квантор в сигнатуре — `ExplicitForAll`.

типизированного λ -исчисления из предыдущей главы. λ -абстракция позволяет обобщать выражения по значениям, каждая абстракция добавляет стрелку в тип выражения, аппликация же снимает стрелку:

$$\frac{x : \tau, \Gamma \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau. M : \tau \rightarrow \sigma} \text{Lam} \quad \frac{\Gamma \vdash M : \tau \rightarrow \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M N : \sigma} \text{App}$$

Полиморфное λ -исчисление (*System F*) добавляет точно такую же пару конструкций, только для типов [Pierce, 2002, глава 23]. λ -абстракция $\Lambda\alpha. M$ обобщает выражение по типу: это функция, аргументом которой является тип. Она добавляет в тип выражения квантор $\forall$ подобно тому, как λ добавляет стрелку¹⁰. Применение терма к типу $M \sigma$ (**универсальная аппликация (universal application)**) снимает квантор: вызывающий код выбирает, с каким конкретным типом σ он будет использовать терм, и σ подставляется в тип вместо α :

$$\frac{\alpha, \Gamma \vdash M : \tau}{\Gamma \vdash \Lambda\alpha. M : \forall\alpha. \tau} \text{TLam} \quad \frac{\Gamma \vdash M : \forall\alpha. \tau}{\Gamma \vdash M \sigma : [\alpha \mapsto \sigma] \tau} \text{TApp}$$

Таким образом, например, функция *id* фактически принимает *два аргумента: тип и значение*:

$$\begin{aligned} id &: \forall\alpha. \alpha \rightarrow \alpha \\ id &= \Lambda\alpha. \lambda x : \alpha. x \\ id \text{ nat} &: nat \rightarrow nat \\ id \text{ nat } 42 &: nat \end{aligned}$$

Сравните с записью на Haskell выше: определение `id @a (x :: a) = x` — это в точности $\Lambda\alpha. \lambda x : \alpha. x$, а вызов `id @Int` — это *id nat*.

В обычном коде на Haskell типовые абстракции и аппликации, как и кванторы в начале типа, расставляются неявно механизмом вывода типов. Достаточно конвенции именования: конкретные типы начинаются с большой буквы, а типовые переменные — с маленькой. Писать их вручную приходится, когда информации из терма недостаточно, чтобы вывести тип, когда нужно задать порядок типовых параметров, или чтобы сослаться на абстрагированный тип в теле функции¹¹.

Упражнение 11 Какой тип у выражения `const @Int`? А у `const @Int @Char`? От чего зависит порядок, в котором передаются типы?

Цена такого полиморфизма на уровне типов невелика: пока кванторы стоят только снаружи типа, алгоритм Хиндли–Милнера выводит все типовые абстракции и аппликации сам. Платить приходится, когда кванторы появляются в других местах (2.1.1), а также во время исполнения (2.2).

¹⁰В теории зависимых типов и стрелка, и квантор $\forall$ являются частными случаями одной конструкции — Π -типа.

¹¹Ссылаться из тела функции на типовые переменные из её сигнатуры позволяет расширение `ScopedTypeVariables`.

Полиморфными бывают не только функции, но и типы данных. В предыдущей главе мы видели, что `Maybe` — это не тип, а типовой конструктор: чтобы получить тип, ему нужно передать аргумент, например, `Maybe Int`. Если ранее мы управляли типом с уровня термов универсальной аппликацией, то теперь мы управляем типом на уровне типов. Иначе говоря, на уровне типов появляется своё маленькое λ -исчисление: λ -абстракция в типах, аппликация в типах и, соответственно, β -редукция.

Для примера определим тип пары по Чёрчу, абстрагированный от конкретных типов компонент, чтобы пользователь мог выбрать нужные ему. Сначала на Haskell, где вычислительную семантику типов можно проследить в синонимах типов:

```
1 type Pair a b = forall c . (a -> b -> c) -> c

2 pair :: a -> b -> Pair a b
3 pair x y = \f -> f x y

4 fst :: Pair a b -> a
5 fst p = p (\x _ -> x)
```

Пара здесь закодирована по Чёрчу: это функция, которая отдаёт обе компоненты любому, кто скажет, что с ними делать. Синоним `Pair` — функция на типах: `Pair Int Int` вычисляется в `forall c . (Int -> Int -> c) -> c`. Обычные конструкторы типов, напротив, номинативны: `(Int, Int)` или `Maybe Int` никуда далее не вычисляются.

А раз на уровне типов есть функции и аппликации, то и типы можно составить неправильно: `Maybe Maybe` или `Int Bool`. Значит, типам нужна своя система типов. Это уже знакомая нам система кайндов: она (пока) представляет собой простейшую “систему типов для типов” и обеспечивает well-formedness типов и строгую нормализуемость¹². Получается двухэтажная конструкция (рис. 5): вершинам дерева терма приписаны типы, но типы — тоже деревья, и уже их вершинам приписаны кайнды. Кайнды, в свою очередь, тоже деревья (например, `Type -> Type`); что приписывается их вершинам, мы увидим в конце этого раздела (рис. 7).

Правила приписывания кайндов устроены в точности как правила *Lam* и *App* для термов, только этажом выше: абстракция по типовой переменной добавляет стрелку в кайнд, аппликация типа к типу её снимает [Pierce, 2002, глава 29]¹³:

$$\frac{\alpha : \kappa_1, \Gamma \vdash \tau : \kappa_2}{\Gamma \vdash \lambda \alpha : \kappa_1. \tau : \kappa_1 \rightarrow \kappa_2} KLam \quad \frac{\Gamma \vdash \tau : \kappa_1 \rightarrow \kappa_2 \quad \Gamma \vdash \sigma : \kappa_1}{\Gamma \vdash \tau \sigma : \kappa_2} KApp$$

Аппликация абстракции к аргументу редуцируется подстановкой, как и на уровне термов:

$$(\lambda \alpha : \kappa. \tau) \sigma \rightsquigarrow [\alpha \mapsto \sigma] \tau$$

Кроме того, нужны правила для конструкций, которые строят обычные типы: стрелка и квантор из типов кайнда * делают тип кайнда *. Квантор при этом связывает типовую переменную

¹²Строгая нормализуемость — любой порядок редукций приводит к нормальной форме.

¹³В формулах кайнд `Type` обозначаем традиционной звёздочкой, как это принято в литературе Pierce [2002].

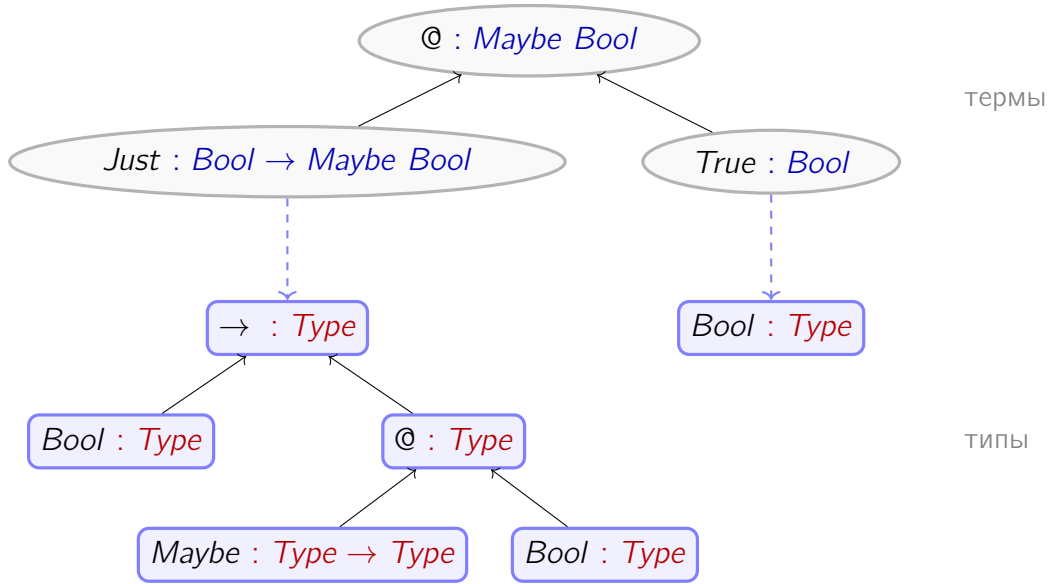


Рис. 5: Два этапа типизации выражения `Just True`. Вершинам дерева терма приписаны типы (синим). Каждый тип сам является деревом (пунктирные стрелки), и его вершинам приписаны кайнды (красным).

любого кайнда:

$$\frac{\Gamma \vdash \tau : * \quad \Gamma \vdash \sigma : *}{\Gamma \vdash \tau \rightarrow \sigma : *} \text{KArrow} \quad \frac{\alpha : \kappa, \Gamma \vdash \tau : *}{\Gamma \vdash \forall \alpha : \kappa. \tau : *} \text{KAll}$$

В Haskell кайнды, как и типы, выводятся автоматически, а посмотреть их можно командой `:k`, как мы делали в предыдущей главе.

В исчислении то же самое записывается с явными абстракциями и аппликациями на обоих уровнях (кайнды связанных типовых переменных указаны верхним индексом):

$$\begin{aligned} \text{Pair} &: * \rightarrow * \rightarrow * \\ \text{Pair} &= \lambda \tau^* \sigma^*. \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma \\ \text{pair} &: \forall \alpha \beta. \alpha \rightarrow \beta \rightarrow \text{Pair } \alpha \beta \\ \text{pair} &= \Lambda \alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda \gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y) \\ \text{fst} &: \forall \alpha \beta. \text{Pair } \alpha \beta \rightarrow \alpha \\ \text{fst} &= \Lambda \alpha^* \beta^*. \lambda p^{\text{Pair } \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta) \end{aligned}$$

Прочитаем определение `fst`. Принимаем два типа и пару p . Пара — полиморфная функция, поэтому сначала выбираем для неё тип результата α , а затем передаём ей функцию $\mathbf{K}$, возвращающую первый из двух аргументов.

Упражнение 12 Постройте по правилам кайндизации дерево вывода $\text{Pair} : * \rightarrow * \rightarrow *$. Какие правила и в каком порядке в нём применяются? Поредуцируйте тип `Pair nat bool` до нормальной формы.

Упражнение 13 Реализуйте на Haskell функции `snd` и `swap :: Pair a b -> Pair b a`. Запишите `snd` в исчислении с явными типовыми абстракциями и аппликациями.

Haskell не позволяет создавать функции на типах по месту с помощью явной типовой лямбды¹⁴ ввиду проблематичности этой конструкции для вывода типов. Однако именованные функции на типах есть, и мы рассмотрим их далее 3.2. В Scala существует нетривиальный трюк¹⁵¹⁶, который позволяет этого добиться. Scala3, однако, включила эту возможность непосредственно в язык¹⁷.

2.1.1 First-class polymorphism

Рассмотрим функцию `f g = (g 42, g 'a')`. Программа корректна, если передать в неё, например, `id`. Однако алгоритм Хиндли–Милнера её отвергает: аргумент функции он считает мономорфным, а `g` применяется и к числу, и к символу. Чтобы протипизировать такую программу, нужно разрешить квантору стоять не только снаружи типа, но и слева от стрелки.

Иначе говоря, нужна возможность писать функции, которые принимают другие полиморфные функции в качестве аргументов. Типы таких функций называются **типами высшего ранга (higher-rank types)**¹⁸. Так, типовой параметр функции `g` определяет функция `f`, а не вызывающий функцию `f`:

```
1 f :: (forall a . a -> a) -> (Int, Char)
2 f g = (g @Int 42, g @Char 'a') -- универсальная аппликация для наглядности
3 ghci> f (\x -> x)
```

Цена типов высшего ранга в том, то есть глобальный вывод типов Haskell в этом случае перестаёт работать. Но если типы высшего ранга приписать вручную, остальной вывод будет работать как раньше. Например, тип числа Чёрча полиморфен, поэтому функции над числами Чёрча имеют высший ранг¹⁹²⁰:

```
1 suc :: (forall a . (a -> a) -> a -> a) -> (b -> b) -> b -> b
2 suc n s z = s (n s z)
```

Упражнение 14 Какую ошибку выдаст GHC, если убрать у функции `f` сигнатуру и типовые аппликации? Какой тип для `g` он пытался вывести?

Упражнение 15 Какой ранг имеет тип `Int -> (forall a . a -> a)`?

¹⁴<https://stackoverflow.com/questions/4069840/lambda-for-type-expressions-in-haskell>

¹⁵(stackoverflow) Scala type lambdas.

¹⁶<https://stackoverflow.com/questions/9443004/what-does-the-operator-mean-in-scala>

¹⁷<https://docs.scala-lang.org/scala3/reference/new-types/type-lambdas.html>

¹⁸В Haskell их включает расширение RankNTypes.

¹⁹<https://okmij.org/ftp/tagless-final/course/Boehm-Berarducci.html>

²⁰Любому интересующемуся языками программирования предлагается провести на сайте Олега Киселёва не один месяц жизни: <https://okmij.org/ftp/README.html>.

От многих проблем сопутствующих типам высших рангов можно избавиться, если создавать для них обёртки. Например, для чисел Чёрча можно создать обёртку `newtype Church`. Теперь код, работающий с обёрткой, может быть протипизирован типами первого ранга, только конструктор имеет тип высшего ранга:

```
1 newtype Church = Church (forall a . (a -> a) -> a -> a)
2 (+) :: Church -> Church -> Church -- rank 1
```

Аналогичный код можно написать и на Java или Kotlin:

```
1 interface Church { fun <a> fold(s: (a) -> a, z: a): a }
2 fun plus(n: Church, m: Church): Church = object : Church {
3     override fun <a> fold(s: (a) -> a, z: a): a = n.fold(s, m.fold(s, z))
4 }
```

По умолчанию типовые параметры можно специализировать только на конкретные типы. Специализацию типовых параметров на полиморфные типы (включающие `forall`'ы внутри себя) называют **импредикативным применением**²¹.

```
1 runST :: (forall s. ST s c) -> c
2 ($) :: forall a b . (a -> b) -> a -> b
3 foo = runST $ ... -- a специализируется на (forall s. ST s c)
```

Этот пример настолько частый, что GHC делает для оператора `$` исключение и типизирует его без расширения. Для любой другой функции с таким же типом, например, собственного оператора аппликации `app`, потребуется само расширение; с ним GHC применяет алгоритм Quick Look, о котором ниже.

Higher-rank типы можно использовать как type-based escape analysis, иначе говоря, не позволять пользователю передавать некоторое значение вовне определённого скоупа. Так, например, Haskell предоставляет эффективную монаду `ST`, позволяющую в рамках ограниченного скоупа работать с мутабельными ячейками памяти Launchbury and Peyton Jones [1995][Maguire, 7.2, ST trick]:

```
1 newtype ST s a = ST (IO a) -- упрощённо, в GHC устроено сложнее
2 runST :: (forall s. ST s a) -> a

3 sumTo :: Int -> Int
4 sumTo n = runST do
5     ref <- newSTRef 0
6     forM [0..n] \i -> modifySTRef ref (+ i)
7     readSTRef ref
```

²¹В Haskell его включает расширение `ImpredicativeTypes`.

Заметим, что если попытаться вернуть из `runST` ссылку на мутабельную ячейку, то тип результата должен будет содержать фантомный параметр `s`. Но в сигнатуре `runST` тип результата `a` стоит вне области действия квантора по `s`: этот квантор связан внутри аргумента, и снаружи `s` просто не существует. Поэтому такая программа не проходит проверку типов:

```
1 newSTRef :: v -> ST s (STRef s v)
2 ghci> runST (newSTRef True) -- ошибка типизации
```

На практике, чтобы отличать такие локально связанные типовые переменные, используют концепцию уровней²² Peyton Jones [2019].

Типы высших рангов вместе с импредикативным применением образуют **полиморфизм первого класса (first-class polymorphism)**, когда полиморфные типы могут использоваться почти так же свободно, как и любые другие. Классический алгоритм глобального вывода Хиндли-Милнера не справляется (и в общем случае задача неразрешима), так что существует большое количество решений, делающих различные компромиссы. Можно сделать вывод типов локальным, опирающемся только на соседние ноды AST и вспомогательные типовые аннотации Pierce and Turner [2000], Christiansen [2013], Dunfield and Krishnaswami [2019]. Либо же можно попытаться помочь глобальному выводу дополнительной предобработкой (Quick Look²³ Serrano et al. [2020], реализованный в GHC начиная с версии 9.2 и включаемый расширением `ImpredicativeTypes`) или дополнительными регулируемыми конструкциями (FreezeML Emrich et al. [2020]).

2.1.2 Higher-order/kinded polymorphism

Функция `fmap` работает одинаково для списков, `Maybe` и деревьев. Чтобы записать её тип, нужно абстрагироваться уже не по типу элементов, а по самому контейнеру, то есть по типовому конструктору кайнда `Type -> Type`.

Haskell позволяет также абстрагироваться по типам произвольных кайндов, а не только `Type`, как в `data` декларациях (**higher-order/kinded types (HKT)**²⁴), так и в полиморфных функциях. Далее мы встретим немало примеров. Так, тип `Fix`, с которым мы подробно познакомимся в 4.3.2, параметризован не типом, а типовым конструктором, и потому имеет кайнд `(Type -> Type) -> Type`. Катаморфизм же абстрагирован по типу стрелочного кайнда:

```
1 newtype Fix f = Fix (f (Fix f))
2 cata :: forall (f :: Type -> Type) a . Functor f => (f a -> a) -> Fix f -> a
```

Упражнение 16 Какой кайнд у `Fix Maybe`? Приведите несколько значений этого типа. Какой знакомый тип они напоминают?

Цена: чтобы вывод типов оставался простым, Haskell требует, чтобы на месте `f` стоял именно типовой конструктор, а не произвольная функция на типах (мы вернёмся к этому

²²<https://okmij.org/ftp/ML/generalization.html>

²³(youtube) A Quick Look at Impredicativity (Simon Peyton Jones)

²⁴<https://serokell.io/blog/kinds-and-hkts-in-haskell>

в 3.2). Во многих же промышленных языках полиморфизма высших порядков нет вовсе, и далее мы рассмотрим технику, позволяющую закодировать его в таком языке (см. 3.1.8).

2.1.3 Обобщённые алгебраические типы данных (GADTs)

Обобщённые алгебраические типы данных (generalized algebraic data types, GADTs) позволяют приписывать данным на уровне типов больше информации. В качестве модельного примера возьмём синтаксис крошечного языка программирования. Зададимся целью не допустить возможности конструирования в Haskell некорректных с точки зрения типов синтаксических деревьев.

```
1 data Expr = Const Int | IsZero Expr | If Expr Expr Expr
```

Сейчас ничто не мешает составить бессмысленную программу вроде `IsZero (IsZero (Const 1))`: некорректная программа на нашем языке является совершенно корректным значением Haskell.

Как мы знаем, конструкторы данных в Haskell — это обычные функции с той лишь разницей, что их реализация генерируется компилятором (аллокация памяти, размещение полей...). У функций есть тип. Например, `IsZero :: Expr -> Expr`.

В Haskell есть синтаксис определения `data` через задание типов конструкторов²⁵. Он совершенно аналогичен рассмотренному ранее, только гораздо более удобен для сложно организованных структур данных. Рассмотренный ранее тип термов `Expr` будет выглядеть следующим образом:

```
1 data Expr where
2   Const :: Int -> Expr
3   IsZero :: Expr -> Expr
4   If :: Expr -> Expr -> Expr -> Expr
```

Для полиморфных структур данных, на примере списка, используется следующий синтаксис. Имя `elem` нужно исключительно для документации и больше никак его использовать нельзя, оно только маркирует наличие типового параметра и позволяет ему вручную задать кайнд²⁶.

```
1 data List (elem :: Type) where
2   Nil :: List a
3   Cons :: a -> List a -> List a
```

Добавим к `Expr` фантомный типовой параметр `ty`, обозначающий тип Haskell, в который должно быть проинтерпретировано данное выражение, и с помощью GADT зададим конкретные значения `ty` результирующим типам конструкторов. Так, мы говорим, что программа сконструированная с помощью `Const` вычисляется в число, `IsZero` вычисляется в булево значение, а условное выражение — в тип веток:

²⁵https://downloads.haskell.org/ghc/latest/docs/users_guide/exprs/gadt_syntax.html#gadt-style

²⁶Кайнд можно не писать. Либо можно не писать имена параметров и просто приписать кайнд типовому конструктору: `data List :: Type -> Type where ...`

```

1 data Expr ty where
2   Const :: Int -> Expr Int
3   IsZero :: Expr Int -> Expr Bool
4   If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty

5 eval :: Expr ty -> ty

```

Теперь мы можем написать безопасный типизированный интерпретатор. Обратите внимание, что при сопоставлении с образцами конструкторов, у нас уточняется информация о типовом параметре.²⁷

```

1 eval :: Expr ty -> ty
2 eval = \case
3   Const x  -> x           -- ty ~ Int
4   IsZero t -> eval t == 0 -- ty ~ Bool
5   If c t e -> if eval c then eval t else eval e

```

Цена: вывод типов при сопоставлении с образцом по GADT без подсказки не работает, поэтому функциям вроде `eval` сигнатура обязательна. Далее мы рассмотрим, как GADT в Haskell выражаются через более базовые механизмы языка и почему вывод типов для них сложнее (3.4.4).

Упражнение 17 Добавьте в язык пары: конструкторы `MkPair :: Expr a -> Expr b -> Expr (a, b)` и `Fst :: Expr (a, b) -> Expr a`. Допишите `eval`. Какие равенства типов возникают в новых ветках?

2.1.4 Структуры на уровне типов, data promotion

Чтобы обрести больший контроль корректности программ, научимся кодировать произвольные структуры данных на уровне типов. В качестве модельной задачи зададим структуру данных, моделирующую вектор, но с контролем длины.

Для начала определим натуральные числа на уровне типов в стиле Пеано:

```

1 data Zero
2 data Suc n

```

Упражнение 18 Сколько обитателей типа `Suc (Suc Zero)`?

Теперь мы можем задать тип вектора, содержащий информацию о длине:

```

1 data Vec (size :: Type) (elem :: Type) where
2   VNil :: Vec Zero a
3   VCons :: a -> Vec n a -> Vec (Suc n) a

```

²⁷Тут используется удобное расширение `LambdaCase`, позволяющее не вводить лишние имена.

```

4 example :: Vec (Suc (Suc Zero)) Int
5 example = VCons 1 (VCons 2 VNil)

```

Для такого типа, например, можно написать безопасную функцию `zip`, работающую только на векторах одинаковой длины:

```

1 vzip :: Vec n a -> Vec n b -> Vec n (a, b)
2 vzip VNil VNil = VNil -- n ~ Zero
3 vzip (VCons x xs) (VCons y ys) = VCons (x, y) (vzip xs ys) -- n ~ Suc n'

```

Заметьте, что в остальных ветках `vzip` должны возникнуть эквивалентности, начинающиеся с различных конструкторов, например, `Zero ~ Suc n`. Поскольку невозможно построить такие аргументы функции, Haskell позволяет соответствующие ветки не рассматривать.

Упражнение 19 Напишите функцию добавления в конец элемента вектора. Двигайтесь последовательно, заполняя типовые дыры и отслеживая возникающие эквивалентности.

Сейчас язык кайндов, который должен контролировать типы, слишком беден. Действительно, кайнд `Suc` — `Suc :: Type -> Type`, соответственно ничто не мешает написать, например, `Suc (Maybe Int)`. В то же время он слишком ограничивающий, поскольку не поддерживает полиморфизм, что дало начало большому количеству дублирований а ля `Typeable (ty :: Type)`, `Typeable1 (ty :: Type -> Type)`...

Современный Haskell позволяет объявлять новые типы и кайнды подобно тому, как `data` позволяет объявлять новые типы²⁸.

```

1 type data Nat = Zero | Suc Nat

```

Теперь вектору можно приписать более точный кайнд:

```

1 data Vec (size :: Nat) (elem :: Type) where
2   VNil :: Vec Zero a
3   VCons :: a -> Vec n a -> Vec (Suc n) a

```

Упражнение 20 Что выведет `ghci> :k Vec`?

Другой вариант добиться того же самого — автоматически продвигать (promotion) все `data` декларации на уровень выше²⁹ Yorgey et al. [2012]. А именно: любой конструктор типа также становится кайндом, а конструктор данных — конструктором типа. Так, в примере с числами, мы можем задекларировать натуральные числа как обычно и использовать на уровне типов:

²⁸Расширение `TypeData`.

²⁹Расширение `DataKinds`.

```

1 data Nat = Zero | Suc Nat
2 ghci> :k Suc -- тут понятно, что Suc используется как тип
3 Suc :: Nat -> Nat

```

Поскольку типы и термы в Haskell живут в разных пространствах имён, можно называть конструкторы типов и данных одинаково. Однако если продвинуть такой тип данных, возникнет неоднозначность: мы имеем в виду тип или продвинутый конструктор. Haskell позволяет указать явно, что речь идёт о продвинутом конструкторе с помощью одинарной кавычки.

```

1 data T = T Nat
2 ghci> :k T
3 T :: Type      -- про конструктор типа
4 ghci> :k 'T
5 'T :: Nat -> T -- про продвинутый конструктор данных

```

Не любые `data` декларации подходят для продвижения, в то же время `type data` декларации позволяют явно запросить структуру уровня типов и получить внятные ошибки, если декларация написана неправильно.

В случае продвижения полиморфного типа, мы получаем полиморфные кайнды³⁰:

```

1 data [a] = [] | (:) a [a]
2 ghci> :k '(:)
3 '(:) :: forall k . k -> [k] -> [k]

```

Term	Type	Kind
<code>Zero</code>	<code>Nat</code>	<code>Type</code>
<code>[Zero, Suc Zero]</code>	<code>[Nat]</code>	<code>Type</code>
<code>[]</code>	<code>forall a. [a]</code>	<code>Type</code>
<code>(:)</code>	<code>forall a. a -> [a] -> [a]</code>	<code>Type</code>
	<code>'Suc 'Zero</code>	<code>Nat</code>
	<code>'['Zero, 'Suc 'Zero]</code>	<code>[Nat]</code>
	<code>'[Int, Double]</code>	<code>[Type]</code>
	<code>'[]</code>	<code>forall k. [k]</code>
	<code>'(:)</code>	<code>forall k. k -> [k] -> [k]</code>

Рис. 6: Пример продвижений в Haskell.

Примеры продвижения различных конструкций можно увидеть в таблице 6.

В качестве примера, зададим гетерогенный список, индексированный типами элементов:

```

1 data HList (tys :: [Type]) where
2   HNil :: HList '[]

```

³⁰Расширение PolyKinds.

```

3   HCons :: ty -> HList tys -> HList (ty ': tys)

4   example :: HList '[Int, Bool, Double]
5   example = HCons 42 $ HCons True $ HCons 12.5 HNil

```

Структуры данных тоже могут быть полиморфными по кайндам. Рассмотрим тип `Tagged`, позволяющий дополнить тип значения дополнительным типовым тегом. Кайнд тега может быть произвольным, поэтому, например, можем использовать встроенные в систему типов строковые и числовые константы³¹ (другой пример использования полиморфных кайндов — тип `Proxy`, см. 2.1.5):

```

1   newtype Tagged (tag :: k) (a :: Type) = Tagged a
2   ghci> :t Tagged
3   Tagged :: forall k (tag :: k) a. a -> Tagged tag a

4   example :: Tagged ("dbId" :: Symbol) Int
5   example = Tagged 42

```

Современный Haskell в итоге пришёл к тому, что система типов не делает различий между типами и кайндами (рис. 7). В частности, `Type :: Type`. Это нужно для расширения возможностей Haskell в сторону программирования с зависимыми типами путём добавления несинтаксических эквивалентностей для кайндов³². Соответствующее расширение *System FC*, внутреннего языка GHC (см. далее 3.4.4), равенствами кайндов описано в работе Weirich et al. [2013]³³³⁴.

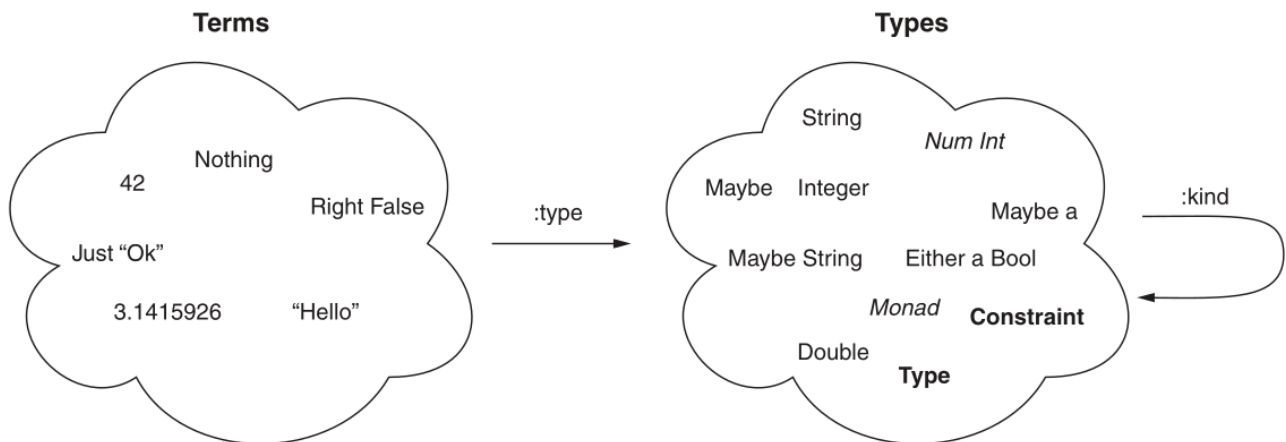


Рис. 7: Типы и кайнды — одно (Bragilevsky).

³¹TypeLits: литералы уровня типов кайндов `Symbol` и `Nat`.

³²Расширение `TypeInType`, ныне часть `PolyKinds`.

³³(youtube) Мини-курс на русском языке про развитие Haskell в сторону зависимой типизации.

³⁴(youtube) Мини-курс на русском языке — система вывода типов Haskell.

2.1.5 Факультативно: эмуляция типовых абстракций и аппликаций (`Proxy`)

Это факультативный и специфичный для Haskell материал. Его можно пропустить при первом чтении и вернуться, когда тип `Proxy` встретится в коде следующих глав.

Проблема здесь историческая: расширения, позволяющие вручную задавать типовые аппликации и абстракции, появились в Haskell сравнительно недавно³⁵. До этого явно передать тип в функцию было нельзя, и пользовались следующей техникой, которая до сих пор встречается во многих библиотеках.

В стандартной библиотеке определён тип `Proxy` с одним параметром. Это **фантомный типовой параметр** — значения соответствующего типа не хранятся в структуре данных, он только позволяет размещать дополнительную информацию на уровне типов³⁶. Соответственно, неинформативную константу `Proxy` можно проаннотировать нужным типом и передать в функцию, чтобы специализировать типовой параметр на нужный тип. Или можно принять `Proxy` и воспользоваться `ScopedTypeVariables` для типовых сигнатур в паттернах³⁷.

```
1 data Proxy a = Proxy

2 id :: Proxy a -> a -> a
3 ghci> :t id (Proxy :: Proxy Int)
4 id (Proxy :: Proxy Int) :: Int -> Int

5 id (Proxy :: Proxy [a]) x = (x :: [a])
```

Иногда прокси-тип оставляют полиморфным, чтобы не зависеть от `Data.Proxy`. Вместо конкретного значения иногда передают специализированное значение $\perp$, а получатель, не зная тип, не сможет его форсировать (однако, любые вхождения $\perp$ в терм слишком раздражают, поэтому это скорее не очень хорошая практика).

```
1 id :: proxy a -> a -> a
2 id (_ :: proxy a) x = (x :: a)

3 ghci> :t id (undefined :: Proxy Int)
4 id (undefined :: Proxy Int) :: Int -> Int
```

2.2 Реализация параметрического полиморфизма

До сих пор полиморфизм был для нас свойством системы типов: мы обсуждали, какие программы он позволяет протипизировать. Теперь посмотрим, сколько он стоит во время исполнения.

Конвенция вызова³⁸ представляет собой набор соглашений между тем как функция ком-

³⁵TypeApplications, TypeAbstractions.

³⁶https://wiki.haskell.org/Phantom_type

³⁷Типовой параметр на самом деле имеет полиморфный кайнд: `data Proxy (a :: k) = Proxy`, чтобы эта техника работала с типами произвольных кайндов (см. 2.1.4).

³⁸https://en.wikipedia.org/wiki/Calling_convention

пилируется и как должна вызываться. Например, функция принимает два аргумента, каждый размером в машинное слово, и возвращает один результат размером в машинное слово. Тогда сгенерированный низкоуровневый код этой функции может, например, ожидать, что оба аргумента передаются через специальную пару регистров, а складывать результат он будет в третий. В таком случае вызывающий код обязан предоставить аргументы в правильных регистрах и ожидать результата в некотором третьем, заранее оговоренном регистре.

В общем случае, конвенция вызова функции зависит от типов аргументов и результата. Нужно знать как минимум их размер, чтобы понять, размещать их в регистрах или на стеке. Нужно знать, это указатель (**reference type**) или значение само по себе (**value type**), чтобы понимать, как с ним работать. В структурах данных нужно знать смещения полей.

Таким образом, реализация параметрического полиморфизма в языке — это не тривиальная задача. Разные языки используют различные подходы, все со своими достоинствами и недостатками.

2.2.1 Мономорфизация

Мономорфизация — самый прямолинейный подход: полиморфные функции и структуры компилируются для каждого набора типовых аргументов. Так, если различных наборов типовых аргументов, с которыми эта функция вызывается, например, 100 (что запросто может быть), то её код будет компилироваться сто раз и занимать в бинарнике в сто раз больше места. Так делают, например, C++ и Rust.

На самом деле всё ещё хуже. Если проект многомодульный и состоит из множества единиц компиляции (кусков, которые компилируются отдельно), то одна и та же специализация функции на типовые аргументы будет компилироваться заново во всех единицах компиляции, где такая специализация нужна. А затем, линкер будет заниматься удалением дубликатов, что тоже не самый быстрый и эффективный процесс.

- + Порождаемый код максимально эффективен для каждого типа;
- + Легко на этапе компиляции отработывают `is`-проверки значений на принадлежность определённому типу (в остальных подходах с этим всё сложно);
- Время компиляции крайне велико;
- Существенно увеличивается размер результирующего бинарного файла, что может быть критично для некоторых приложений;
- Может неэффективно работать из-за засорения кеша кода в процессоре;
- В интерфейсах не может быть полиморфных методов, так как мы не знаем в месте вызова, к какому именно наследнику относится вызываемый метод, и какой код нужно специализировать (аналогично, не работает `higher-rank` полиморфизм);
- К полиморфным функциям нельзя динамически линковаться (у них нет кода до специализации);
- В общем случае нельзя поддержать `variance`, потому что код компилируется для конкретного типа и в общем случае не может работать для произвольного подтипа или

супертипа (если `reference` и `value` типы могут находиться в одной иерархии подтипизации).

Некоторые языки (например, C++ и Zig) не делают инстанциацию скрытой деталью реализации языка, а предоставляют её как инструмент пользователям. Это даёт следующие возможности:

- Если разрешить использовать значения в типах, инстанциация может использоваться как механизм вычислений на этапе компиляции.
- Если отложить проверку ошибок на стадию инстанцирования, то мы получим своего рода статическую утиную типизацию. Это позволит не описывать сложные сигнатуры полиморфных функций. Однако тогда функции для тестирования придётся вручную инстанцировать для всевозможных типов, иначе нельзя будет статически понять, компилируется ли она хотя бы для этих типов.

2.2.2 Стирание типа

Можно всё сделать наоборот, унифицировав значения, которые приходят на вход полиморфным функциям и хранятся в полиморфных структурах данных, вместо того, чтобы компилировать код под каждый тип.

Пусть каждое значение будет аллоцировано в куче и передаваться по указателю. Тогда мы сможем переиспользовать один и тот же код для разных типовых аргументов — он просто будет ожидать указатели.

- + Каждая функция компилируется ровно один раз — быстро;
- + Можно динамически загружать новые полиморфные функции и типы и использовать их друг с другом;
- + Гибкость — вариантность, полиморфные методы в интерфейсах, `higher-rank types` и т.д. просто работают;
- Аллокация в куче и разыменование указателя может очень сильно замедлить код;
- Поскольку информация о типах стирается, нельзя ничего сделать с типовым аргументом, не имея его обитателей (например, запросить рефлексией информацию или сделать `is` проверку).

Такого подхода придерживаются JVM, Haskell и, как правило, другие функциональные языки ввиду его гибкости и скорости компиляции.

Особую проблему вызывает работа с примитивами и другими `value`-типами, потому что каждое значение приходится сначала боксить (переносить в кучу), а потом уже использовать в полиморфном контексте. Поэтому языки борются с этим как могут. Некоторые языки урезают диапазоны значений примитивов, чтобы зарезервировать бит, определяющий, это указатель или значение. Код консультируется с этим битом для работы (похоже на 2.2.4).

```

struct value_witness_table {
    size_t size, align;
    void (*copy_init)(opaque *dst, const opaque *src, type *T);
    void (*copy_assign)(opaque *dst, const opaque *src, type *T);
    void (*move_init)(opaque *dst, opaque *src, type *T);
    void (*move_assign)(opaque *dst, opaque *src, type *T);
    void (*destroy)(opaque *val, type *T);
};

```

Рис. 8: Swift value witness table.

Так делают, например, OCaml и Koka. Агрессивный инлайнинг вместе с другими оптимизациями тоже помогают Blanchet [1999]. Java, например, также пытается аккуратно двигаться в сторону возможности мономорфизации³⁹⁴⁰.

2.2.3 Гибридный подход

C# реализует гибридный подход⁴¹. Они различают значения, хранимые в куче — reference types, и значения, хранимые на стеке — value types. Для первых они генерируют одну специализацию, работающую с указателями. Для каждого набора value-типов они генерируют лениво, в рантайме, специализации.

То есть следы дженериков в таком подходе есть и в промежуточном представлении CIL, и в рантайме.

- + value-типы хранятся и передаются as-is без боксинга;
- + Доступна рефлексия по дженерикам;
- + Небольшое время компиляции;
- Инстанциация в рантайме замедляет исполнение;
- Variance работает только для reference types (что странно — есть “правильная” подтипизация, а есть “неправильная”).

2.2.4 Использование виртуальной таблицы свойств типов

Swift⁴² вместе с каждым типовым параметром передаёт value witness table (рис. 8). Это таблица со всей необходимой информацией о типе: размер и выравнивание, что нужно сделать при копировании и перемещении объекта (например, инкрементировать счётчик ссылок). Таким образом, скомпилированный код постоянно обращается к этой таблице и делает виртуальные вызовы функций из неё (рис. 9).

- + Небольшое время компиляции;
- + Предсказуемая эффективность (не приводит к неожиданным паузам в рантайме);

³⁹Type Specialization of Java Generics - What If Casts Have Teeth?

⁴⁰<https://cr.openjdk.org/~jrose/values/parametric-vm.html>

⁴¹Generics in the runtime (C# programming guide).

⁴²(youtube) 2017 LLVM Developers' Meeting: “Implementing Swift Generics”

```

Example:
func f<T>(_ t: T) -> T {
    let copy = t
    return copy
}

Implementation:
void f(opaque *result, opaque *t, type *T) {
    opaque *copy = alloca(T->vwt->size);
    T->vwt->copy_init(copy, t, T);
    T->vwt->move_init(result, copy, T);
    T->vwt->destroy(t, T);
}

```

Рис. 9: Код полиморфной функции, порождаемый компилятором Swift.

- + Эффективная работа с value-значениями;
- + Высокая гибкость;
- + Информация о типах не стирается;
- Серьёзный константный оверхед на динамические вызовы через таблицу, эффективность очень сильно зависит от компиляторных оптимизаций.

Это своего рода реализация параметрического полиморфизма через специальный. Вместе с типом в функцию неявно передаётся таблица операций, которые умеет этот тип: узнать размер, скопировать, переместить. В следующей главе мы увидим, что ровно так же, передачей словарей функций, реализуются классы типов (3.1.1), только набор операций там задаёт программист, а не компилятор.

2.2.5 Сравнение подходов

Сведём четыре подхода в таблицу 10. Ни один из них не выигрывает по всем строкам: каждый язык выбирает, чем платить.

Упражнение 21 В Kotlin (JVM, стирание типов) проверка `x is T` для типового параметра `T` разрешена только внутри `inline`-функций, где параметр помечен как `reified`. Объясните это ограничение. Какой из рассмотренных подходов к реализации полиморфизма здесь локально применяется?

Упражнение 22 В Rust (мономорфизация) трейт с обобщённым методом нельзя использовать как `trait object` (`dyn Trait`). Каким из перечисленных недостатков мономорфизации это объясняется?

Упражнение 23 Приложение подгружает плагин, скомпилированный отдельно, и хочет вызвать из него обобщённую функцию с типом, о котором автор плагина не знал. Для каждого из четырёх подходов определите, возможно ли это, и почему.

	Мономорфизация (2.2.1)	Стирание типа (2.2.2)	Гибридный подход (2.2.3)	Таблица свойств типа (2.2.4)
Языки	C++ , Rust	JVM, Haskell	C#	Swift
Скорость кода	+ лучшая	— боксинг, индирекции	+ для value-типов	— вызовы через таблицу
Время компиляции, размер бинарного файла	—	+	+	+
Динамическая загрузка полиморфного кода	—	+	+	+
Полиморфные методы в интерфейсах, higher-rank	—	+	+	+
Тип известен во время исполнения	— только статически	— стёрт	+	+

Рис. 10: Сравнение реализаций параметрического полиморфизма.

2.3 Полиморфизм по конвенции вызова

Как мы уже обсуждали выше 2.2.2, параметрический полиморфизм в Haskell реализуется следующим образом: все значения хранятся в куче и передаются в полиморфные функции по указателю. Однако, если для вычислительного кода важна производительность, такой подход не годится ввиду большой нагрузки на подсистему управления памятью и множества индирекций. Поэтому Haskell позволяет также писать код с использованием `unboxed` значений. А если конвенция вызова не принципиальна, можно по ней абстрагироваться и писать один код для `boxed` и `unboxed` значений Eisenberg and Peyton Jones [2017].

2.3.1 Разновидности runtime представлений в Haskell

На рисунке 11 можно увидеть классификацию значений в Haskell с примерами типов. **Unboxed типы** — их значения удерживаются и передаются по значению. **Boxed**, соответственно, наоборот, передаются по указателю и хранятся в куче. Обычный `Int` является просто декларацией следующего вида, где `I#` — это обычный конструктор с необычным именем, содержащий `unboxed` значение.

```
1 data Int = I# Int#
```

	Boxed	Unboxed
Lifted	<i>Int</i> <i>Bool</i>	
Unlifted	<i>ByteArray</i> _#	<i>Int</i> _# <i>Char</i> _#

Рис. 11: Виды значений в Haskell с примерами Eisenberg and Peyton Jones [2017].

Lifted типы — содержат $\perp$ в качестве значения. Иначе говоря, могут содержать отложенные вычисления (это для них специальным образом обеспечивают компилятор и рантайм). **Unlifted типы** — наоборот, не могут быть отложенными. Операции, производящие значения unlifted типов всегда энергичные. Свойство lifted/unlifted называют **levity**. Чтобы распространить дальнейшее изложение на энергичные языки, можно levity заменить на boxity и всё останется справедливым.

в именах типов и функций — это конвенция, показывающая, что где-то рядом происходит работа с unlifted значениями⁴³.

Также в Haskell есть unboxed кортежи, которых не существует на этапе исполнения. Например, следующая функция как бы возвращает пару значений, но в действительности компилятор может их разместить, например, в паре регистров. Соответственно, паттерн-матчинг по таким кортежам, просто позволяет сослаться на каждое из этих значений.

```
1 divMod# :: Int -> Int -> (# Int, Int #)
2 case divMod# n k of (# quot, rem #) -> ...
```

Соответственно, во время исполнения нет никакого различия между по-разному вложенными unboxed кортежами, все они представлены одним и тем же набором значений:

```
1 (# A, (# B, C #) #) ≡ (# (# A, B #), C #) ≡ (# A, B, C #)
```

2.3.2 Классификация значений по runtime представлению

Значения различных типов могут быть на этапе исполнения устроены по-разному. То есть нам нужна некоторая система классификации типов. Но такая система в Haskell уж есть — кайнды. Опишем в виде структур данных предметную область, а потом продвинем на нужный уровень (2.1.4).

Стандартная библиотека Haskell предоставляет следующие типы данных⁴⁴:

```
1 TYPE :: RuntimeRep -> Type
```

⁴³Нужно подключить расширение MagicHash, чтобы пользоваться *#* в идентификаторах.

⁴⁴https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/representation_polymorphism.html

```

2 data Levity = Lifted | Unlifted

3 data RuntimeRep = BoxedRep Levity
4                 | IntRep | DoubleRep
5                 | TupleRep [RuntimeRep]
6                 | SumRep [RuntimeRep]
7                 | ...

8 type LiftedRep = BoxedRep Lifted

9 type Type = TYPE LiftedRep

```

`TYPE` — это магический тип, определённый в компиляторе. Он параметризован runtime-представлением значений. Теперь привычный `Type` — это частный случай с boxed lifted значениями.

- `Int :: TYPE (BoxedRep Lifted)` или `:: Type`
 - `IntRep` и `DoubleRep` соответствуют представлению численных констант (в зависимости от архитектуры процессора, целые числа и числа с плавающей запятой может быть необходимо располагать в различных специальных регистрах)
`Int# :: TYPE IntRep`
 - `Maybe Int :: Type`
 - `Maybe :: Type -> Type`
 - `TupleRep` и `SumRep` — unboxed алгебраические типы, представления параметризованы представлениями хранимых значений
`(# Int, Bool #) :: TYPE (TupleRep '[LiftedRep, LiftedRep])`
 - Хотя во время исполнения вложенность кортежей неразличима, на уровне кайндов она для простоты сохраняется, и типы по-разному вложенных кортежей не унифицируются
- ```

1 (# Int#, (# Int, Double# #) #)
2 :: TYPE (TupleRep '[IntRep, TupleRep '[LiftedRep, DoubleRep]])

```

**Упражнение 24** Какой кайнд у типа `(# Int#, Bool #)`? Проверьте себя в `GHCI`, включив расширения `MagicHash` и `UnboxedTuples`.

### 2.3.3 Representation polymorphism

Выставив runtime-представление в структуре кайндов, мы теперь можем параметризоваться по ним. Например, кайнд функциональной стрелки выглядит следующим образом<sup>45</sup>:

```

1 ghci> :k (->)
2 (->) :: forall {q :: RuntimeRep} {r :: RuntimeRep}. TYPE q -> TYPE r -> Type

```

<sup>45</sup>Выключить упрощения: `ghci> :set -fprint-explicit-foralls -fprint-explicit-runtime-reps`

**Упражнение 25** Подумайте, почему функция имеет *boxed* тип. Может ли быть иначе? Может ли это быть полезным?

К сожалению, Haskell выставляет довольно строгое ограничение: связыватели не могут иметь тип, полиморфный по runtime представлению. Можно легко предположить, почему, — нельзя сгенерировать код функции для работы с параметром произвольного рантайм-представления. Это можно решить только мономорфизацией 2.2.1, но Haskell избегает этого подхода<sup>46</sup>. Сообщество также пытается найти другие решения<sup>47</sup> (что-то вроде 2.2.4).

Например, изначально оператор аппликации был обобщён только по возвращаемому типу. Это не порождает проблем, так как вызывающий код сможет вывести представление и сгенерировать подходящий код:

```
1 ($) :: forall r a (b :: TYPE r). (a -> b) -> a -> b
2 f $ x = f x
```

Однако, было замечено, что для оператора аппликации можно получить другую реализацию, не использующую levity-полиморфное связывание<sup>48</sup>:

```
1 ($) :: forall ra rb (a :: TYPE ra) (b :: TYPE rb). (a -> b) -> a -> b
2 ($) f = f
```

**Упражнение 26** Функция *error* полиморфна по представлению результата:

```
1 error :: forall (r :: RuntimeRep) (a :: TYPE r). String -> a
```

Почему это не нарушает ограничение на связыватели?

Таким образом, в Haskell полиморфизм по представлениям несколько вырожден и помогает лишь в небольшом количестве случаев, однако немаловажных. Если позволить мономорфизацию по *RuntimeRep* параметрам, получится система аналогичная гибридной реализации параметрического полиморфизма 2.2.3, только с большим контролем со стороны программиста над мономорфизацией.

---

<sup>46</sup><https://gitlab.haskell.org/ghc/ghc/-/issues/14917>

<sup>47</sup><https://mail.haskell.org/pipermail/haskell-cafe/2023-January/135770.html>

<sup>48</sup>[https://gitlab.haskell.org/ghc/ghc/-/merge\\_requests/10131](https://gitlab.haskell.org/ghc/ghc/-/merge_requests/10131)

### 3 Специальный (ad-hoc) полиморфизм

Как-то Joe Fasel в разговоре с Philip Wadler высказал идею того, что перегрузка функций (overloading) должна находить своё отражение в типах. Wadler понял его неправильно Hudak et al. [2007]. Но то, что он понял, — оказалось классами типов Wadler and Blott [1989].

Christopher Strachey ввёл классификацию полиморфизма на две категории Strachey [2000]. Параметрический — один и тот же код работает с данными различных типов. **Специальный (ad-hoc) полиморфизм** — код выбирается в зависимости от типа. Например, один и тот же символ умножения по-разному действует на целые числа и на числа с плавающей точкой.

Перегрузка в языках обозначает возможность назвать несколько функций с различными наборами входных параметров одинаково. В месте вызова компилятор статически определяет по типам аргументов, какую из них действительно следует вызвать.

```
1 std::string toString(int x) { ... }
2 std::string toString(std::string fmt, double d) { ... }
```

Классы типов обязуют сначала задекларировать именованную сущность (собственно, класс типов), включающую в себя пачку деклараций функций, которые могут быть перегружены для различных типов.

```
1 class Show a where
2 show :: a -> String

3 instance Show Int where
4 show :: Int -> String
5 show = ...
```

Необходимо заметить, что декларация класса типов содержит формальный типовой параметр, по вхождению которого в тип функции, собственно, выбирается перегрузка. Таких параметров может быть много<sup>49</sup>, они могут иметь стрелочные кайнды. Например, в случае класса типов **Applicative**, выбор реализации операции **pure** будет происходить по типовому конструктору результата, то есть даже не по полноценному типу.

```
1 class Functor f => Applicative (f :: Type -> Type) where
2 pure :: a -> f a
3 ...

4 instance Applicative Maybe where
5 pure :: a -> Maybe a
6 ...
```

Также, в отличие от перегрузки, классы типов совместимы с параметрическим полиморфизмом. Так, в типе полиморфной функции нельзя указать, что для типа должна присутствовать определённая перегрузка. Классы типов же позволяют ограничить набор возможных типовых аргументов теми, для которых реализован инстанс нужного класса типов:

---

<sup>49</sup>Расширение MultiParamTypeClasses.

```
1 showPrefixed :: Show a => a -> String -> String
```

Если сравнивать классы типов с переопределением (overriding) в ООП языках, то разрешение вызова виртуальной функции происходит с использованием таблицы, хранящейся в объекте первого параметра (получателя вызова, receiver). Классы типов же опираются исключительно на тип, поэтому, например, возможно определение констант в классах типов<sup>50</sup>:

```
1 class Enum a => Bounded a where
2 minBound :: a
3 maxBound :: a
```

В то же время, классы типов не являются типами, а, скорее, предикатами на типах. Тип удовлетворяет такому предикату, или свойству, если для него есть соответствующий инстанс. Поэтому, в частности, привычный способ в ООП создать гетерогенную коллекцию элементов, имеющих общий интерфейс, напрямую не сработает с классами типов. Например, такой тип не будет корректным: `[Show]`. Мы вернёмся к этой проблеме в 3.4.2.

В этой главе мы сначала разберём, как классы типов устроены внутри: это словари функций, которые компилятор передаёт неявно и конструирует автоматически. Затем посмотрим, как та же идея выбора по типу переносится с кода на сами типы (семейства) и на ограничения (кайнд Constraint). В завершение рассмотрим, какие возможности языка и полезные техники выражают через классы типов.

## 3.1 Классы типов в языке

Несмотря на поразительное могущество, идея реализации классов типов крайне проста. Она была уже во всей полноте представлена в первой работе Wadler and Blott [1989]. В дальнейших работах уточнялся механизм вывода типов в виде сведения к классической системе типов в стиле Hindley-Milner Hall et al. [1996]. Остальные работы, в основном, предлагают огромное разнообразие различных расширений и приложений Jones et al. [1997].

### 3.1.1 Словари

Рассмотрим идею реализации классов типов на примере полиморфной сортировки. Сортировка для списка элементов конкретного типа пишется тривиально:

```
1 sort :: [Int] -> [Int]
2 sort = \case [] -> []; x:xs -> insert x (sort xs)
3 where
4 insert x xs = let (l, r) = List.partition (< x) xs in l ++ x : r
```

---

<sup>50</sup>Современные ООП языки, тем не менее, стремятся поддержать статические члены в интерфейсах, что делает их ближе к классам типов и позволяет делать похожие вещи. Например, статические требования в протоколах Swift (<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/protocols/>) или статические абстрактные члены интерфейсов в C# (<https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/static-virtual-interface-members>).

В реализации единственная информация о типе, которой мы пользуемся — порядок на его обитателях. Таким образом, при переходе к полиморфной сортировке, нам нужно принять словарь с предикатами, задающими нужный порядок для данного типа.

```
1 data OrdDict a = OrdDict { less :: a -> a -> Bool }

2 sort :: OrdDict a -> [a] -> [a]
3 sort d@OrdDict{ less } = \case [] -> []; x:xs -> insert x (sort d xs)
4 where
5 insert x xs = let (l, r) = List.partition (less x) xs in l ++ x : r
```

Теперь, чтобы воспользоваться сортировкой на списке чисел, нужно сконструировать нужный рекорд и вызвать с ним функцию на списке конкретных типов:

```
1 intOrd :: OrdDict Int
2 intOrd = OrdDict { less = (<) }

3 ghci> sort intOrd [3, 2, 1]
4 [1,2,3]
```

Возможна ситуация, когда инстанс для одного типа зависит от инстанса для другого. Например, порядок на списках можно получить автоматически, зная порядок на элементах. В случае словарей мы это моделируем функцией между словарями:

```
1 listOrd :: OrdDict a -> OrdDict [a]
2 listOrd d = OrdDict { less = ... less d ... }
```

Теперь мы можем сортировать список списков, конструируя нужный словарь:

```
1 ghci> sort (listOrd intOrd) [[3, 2], [2, 1], [0]]
2 [[0], [2,1], [3,2]]
```

**Упражнение 27** *Допишите `listOrd` так, чтобы списки сравнивались лексикографически.*

Сравнение явной передачи словарей и классов типов можно увидеть в следующей таблице:

1. Определение словаря функций

```
1 data MyOrd a = MyOrd
2 { less :: a -> a -> Bool }
```

2. Экземпляр словаря для конкретного типа

- Именованное значение

```
1 intMyOrd :: MyOrd Int
2 intMyOrd = MyOrd { less = (<) }
```

3. Явный параметр функции

```
1 sort :: MyOrd a -> [a] -> [a]
```

4. Передаётся пользователем

```
1 test = sort intMyOrd [3, 2, 1]
```

1. Определение класса типов

```
1 class MyOrd a where
2 less :: a -> a -> Bool
```

2. Объявление типа представителем класса типов

- Не имеет имени

```
1 instance MyOrd Int where
2 less = (<)
```

3. Неявный параметр функции

```
1 sort :: MyOrd a => [a] -> [a]
```

4. Передаётся компилятором

```
1 test = sort [3, 2, 1]
```

Таким образом, словарь — это **свидетель (witness)** или доказательство того, что тип удовлетворяет ограничению.

**Упражнение 28** Какой словарь будет соответствовать *higher-kinded* классу типов **Functor**?

### 3.1.2 Неявные аргументы

Можно думать так, что слева от `=>` передаются неявные аргументы функций, выводимые компилятором из контекста. То есть, например, не стоит удивляться вхождению `=>` в тип аргумента, это просто функция с неявным аргументом. Так, следующий код не скомпилируется, потому что в месте использования переменной `x` нет значения типа `Show b`:

```
1 f :: (Show b => b) -> b
2 f x = x -- ошибка
```

Можно это значение принять в функции `f`, тогда оно автоматически пропагируется в `x`:

```
1 f :: Show b => (Show b => b) -> b
2 f x = x
```

Haskell даёт возможность делать неявными и обычные аргументы функции<sup>51</sup>. Фактически, это реализация динамического связывания в статическом языке Lewis et al. [2000] (см. далее 5.3.1). Неявные аргументы берутся из скоупа по имени и подставляются автоматически:

```
1 sortBy :: (a -> a -> Bool) -> [a] -> [a]

2 sort :: (?cmp :: a -> a -> Bool) => [a] -> [a]
3 sort = sortBy ?cmp
```

---

<sup>51</sup>Расширение `ImplicitParams`.

Haskell также предоставляет возможность сохранять словари в структуры данных:

```
1 data ShowDict a where
2 ShowDict :: Show a => ShowDict a

3 f :: ShowDict b -> (Show b => b) -> b
4 f d x = case d of ShowDict -> [x] -- в области видимости доступен инстанс Show b
```

**Упражнение 29** Аргумент  $x$  имеет тип `Show b => b`, то есть ждёт словарь. В какой момент он его получает? Возможна ли именно такая семантика в энергичном языке? Почему?

### 3.1.3 Вывод инстансов

Чтобы вызвать ограниченно-полиморфную функцию, GHC производит вывод инстансов или, иначе говоря, автоматически конструирует свидетелей. Вывод инстансов тесно интегрирован с общей системой вывода типов Haskell Peyton Jones [2019].

В действительности вывод инстансов — это ни что иное, как *задача населения типа*. Действительно, после трансляции в Core (промежуточное представление в GHC), классы типов представляют собой словари функций. У нас в контексте имеются конкретные словари и функции, позволяющие из одних словарей получать другие. Требуется найти терм, конструирующий словарь нужного типа.

Пусть, например, внутри функции `f :: Show a => ..` происходит вызов ограниченно-полиморфной функции `g :: Show [a] => ..`. То есть у нас имеется словарь `d1 :: ShowDict a`, а также функция `d2 :: ShowDict a -> ShowDict [a]`, пришедшая из импортов<sup>52</sup>. Необходимо сконструировать терм типа `ShowDict [a]`. Очевидно, это будет просто аппликация одного к другому: `d2 d1`.

Вывод инстансов происходит рекурсивно. Чтобы вывести `ShowDict [a]`, выводится сначала посылка `ShowDict a`. То есть получается рекурсия по структуре типа. Иначе говоря, вывод инстансов можно эксплуатировать как вычислительный примитив уровня типов. Так, например, мы можем опускать информацию из типов в термы (аналогично `GHC.TypeLits`)<sup>53</sup>:

```
1 type data Nat = Zero | Suc Nat

2 class KnownNat (n :: Nat) where
3 natVal :: Int

4 instance KnownNat Zero where
5 natVal = 0

6 instance KnownNat n => KnownNat (Suc n) where
```

<sup>52</sup>Инстансы можно импортировать пустым импортом: `import Module ()`.

<sup>53</sup>Тип метода `natVal` не упоминает `n`, поэтому вызвать его можно только с явной типовой аппликацией. Такие сигнатуры разрешает расширение `AllowAmbiguousTypes`.

```

7 natVal = 1 + natVal @n

8 ghci> natVal @(Suc (Suc Zero))
9 -- выведется natVal {knownSuc (knownSuc knownZero)}

```

В общем случае процесс населения типа, как можно предположить по вычислительной аналогии, неразрешим. Поэтому GHC накладывает большое количество ограничений на вид инстансов, которые гарантируют тотальность вывода. Подробно эти ограничения описаны в Sulzmann et al. [2007]. Также GHC предоставляет различные расширения, ослабляющие эти ограничения и перекалывающие часть ответственности на плечи программиста<sup>54</sup>. Если отказаться от гарантии завершимости вывода<sup>55</sup>, можно легко написать разворот списка типов на этапе компиляции, как и любую другую функцию. Пусть класс `ShowT` (`tys :: [Type]`) с методом `showTypes :: String` умеет печатать список типов (его реализацию оставим упражнением, см. 3.4.6):

```

1 class Reverse (acc :: [Type]) (tys :: [Type]) where
2 showReverse :: String

3 instance ShowT acc => Reverse acc '[] where
4 showReverse = showTypes @acc

5 instance Reverse (ty : acc) tys => Reverse acc (ty : tys) where
6 showReverse = showReverse @(ty : acc) @tys

7 ghci> showReverse @'[] @'[Char, Int, Double]
8 "Double Int Char"

```

Можно заметить, что процесс вывода классов типов очень похож на вычисление логических программ, например, на Prolog, только без backtracking'a (перебора различных вариантов решений в поисках подходящего). Как, впрочем, и вывод типов в Haskell Peyton Jones [2019] в целом: собранные по программе эквивалентности можно рассматривать как логическую программу, решение этой системы типовых уравнений — как исполнение этой программы.

Между классами типов и выводом типов существует интересная синергия (рис. 12)<sup>56</sup>. Исходя из термов, выводятся типы. Затем, исходя из типов, выводятся инстансы классов типов. То есть мы пишем какой-то интересный интеллектуальный код, а параллельно с нами компилятор выписывает неинтересный код.

Вывод инстансов опирается только на вид “головы” декларации — справа от `=>`, а ограничения слева применяются постфактум. Это можно использовать, чтобы писать более об-

<sup>54</sup>[https://downloads.haskell.org/ghc/latest/docs/users\\_guide/exts/instances.html](https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/instances.html)

<sup>55</sup>Расширение `UndecidableInstances`.

<sup>56</sup>(youtube) Hackett: a metaprogrammable Haskell.

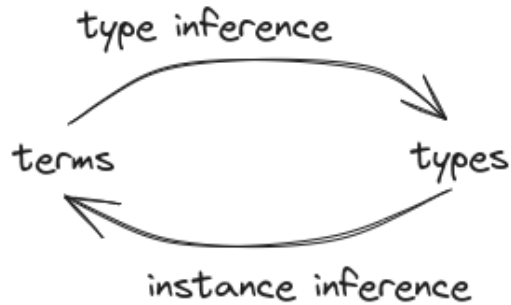


Рис. 12: Классы типов + вывод инстансов = кодогенерация.

щие инстансы. Так, например, работает **constraint trick**<sup>57</sup>, позволяющий разрешать ad-hoc-полиморфные функции в параметрически-полиморфном контексте.

### 3.1.4 Построение типа по значению

После того как мы научились опускать значения из типов, закономерно научиться обратному — поднимать значения в типы. Воспользуемся техникой, описанной в Kiselyov and Shan [2004]. Существует соответствующая библиотека Data.Reflection (см. далее 3.4.7).

В действительности мы, конечно, не можем честно получить синтаксически тип нужного размера, просто потому, что типы существуют строго до стадии исполнения. Однако, как мы знаем, словари классов типов имеют воплощение в рантайме (случай полиморфной рекурсии — как раз пример, когда этого нельзя полностью избежать). Поэтому воспользуемся continuation-passing style, который будет подробно рассмотрен далее в главе 6: вместо того чтобы вернуть результат, примем продолжение, умеющее работать с любым типом с **KnownNat** (пользуемся типовыми абстракциями и аппликациями, см. 2.1)<sup>58</sup>:

```

1 reify :: Int -> (forall n. KnownNat n => a) -> a
2 reify n k
3 | n <= 0 = k @Zero
4 | otherwise = reify (n - 1) \ @n' -> k @(Suc n')
```

Продолжение, передаваемое в рекурсивный вызов, захватывает словарь для типа `n` и конструирует словарь для `Suc n`.

Наконец, можем написать следующую удивительную тождественную функцию, поднимающую сначала значение в тип, а потом опускающее тип обратно в термы:

```

1 wonderId :: Int -> Int
2 wonderId n = reify n (\ @t -> natVal @t)
```

<sup>57</sup><https://chrisdone.com/posts/haskell-constraint-trick/>

<sup>58</sup>В не самых свежих версиях GHC потребуется воспользоваться техникой **Proxy** из 2.1.5.

**Упражнение 30** Что вернёт `wonderId` ( $-3$ )? Является ли `wonderId` тождественной функцией?

*To be continued...*

## Список литературы

- Christopher Strachey. Fundamental concepts in programming languages. *Higher-order and symbolic computation*, 13:11–49, 2000. URL <https://facweb.cdm.depaul.edu/smitsch/courses/csc447fa23/assets/articles/strachey-fundamental-concepts-in-programming-languages.pdf>.
- Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys (CSUR)*, 17(4):471–523, 1985. URL <https://doi.org/10.1145/6041.6042>.
- Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.
- John Launchbury and Simon L Peyton Jones. State in haskell. *Lisp and symbolic computation*, 8(4): 293–341, 1995. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/state-lasc.pdf>.
- Sandy Maguire. *Thinking with Types, Type-Level Programming in Haskell*. URL <https://leanpub.com/thinking-with-types/>
- Simon Peyton Jones. Type inference as constraint solving: how ghc's type inference engine actually works. Keynote talk at Zurihac 2019, 2019. URL <https://simon.peytonjones.org/type-inference/>.
- Benjamin C Pierce and David N Turner. Local type inference. *Acm transactions on programming languages and systems (toplas)*, 22(1):1–44, 2000. URL <https://doi.org/10.1145/345099.345100>.
- David Raymond Christiansen. Bidirectional typing rules: A tutorial. 2013. URL <https://davidchristiansen.dk/tutorials/bidirectional.pdf>.
- Jana Dunfield and Neelakantan R Krishnaswami. Sound and complete bidirectional typechecking for higher-rank polymorphism with existentials and indexed types. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–28, 2019. URL <https://doi.org/10.1145/3290322>.
- Alejandro Serrano, Jurriaan Hage, Simon Peyton Jones, and Dimitrios Vytiniotis. A quick look at impredicativity. *Proceedings of the ACM on Programming Languages*, 4(ICFP):1–29, 2020. URL <https://www.microsoft.com/en-us/research/publication/a-quick-look-at-impredicativity/>.
- Frank Emrich, Sam Lindley, Jan Stolarek, James Cheney, and Jonathan Coates. Freezeml: Complete and easy type inference for first-class polymorphism. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 423–437, 2020. URL <https://link.springer.com/article/10.1208/s12249-010-9382-3>.

- Brent A Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. Giving haskell a promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation*, pages 53–66, 2012. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/p53-yorgey.pdf>.
- Stephanie Weirich, Justin Hsu, and Richard A Eisenberg. System fc with explicit kind equality. *ACM SIGPLAN Notices*, 48(9):275–286, 2013. URL <https://doi.org/10.1145/2544174.2500599>.
- Vitaly Bragilevsky. *Haskell in Depth*. Manning. URL <https://www.manning.com/books/haskell-in-depth>.
- Bruno Blanchet. Escape analysis for object-oriented languages: application to java. *Acm Sigplan Notices*, 34(10):20–34, 1999. URL <https://doi.org/10.1145/320385.320387>.
- Richard A Eisenberg and Simon Peyton Jones. Levity polymorphism. *ACM SIGPLAN Notices*, 52(6):525–539, 2017. URL <https://doi.org/10.1145/3140587.3062357>.
- Paul Hudak, John Hughes, Simon Peyton Jones, and Philip Wadler. A history of haskell: being lazy with class. In *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, pages 12–1, 2007. URL [https://www.researchgate.net/profile/Simon-Peyton-Jones/publication/221501761\\_A\\_history\\_of\\_Haskell\\_Being\\_lazy\\_with\\_class/links/0c960517e31f50f743000000/A-history-of-Haskell-Being-lazy-with-class.pdf](https://www.researchgate.net/profile/Simon-Peyton-Jones/publication/221501761_A_history_of_Haskell_Being_lazy_with_class/links/0c960517e31f50f743000000/A-history-of-Haskell-Being-lazy-with-class.pdf).
- Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 60–76, 1989. URL <https://doi.org/10.1145/75277.75283>.
- Cordelia V Hall, Kevin Hammond, Simon L Peyton Jones, and Philip L Wadler. Type classes in haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 18(2):109–138, 1996. URL <https://doi.org/10.1145/227699.227700>.
- Simon Peyton Jones, Mark Jones, and Erik Meijer. Type classes: an exploration of the design space. In *In Haskell Workshop*, 1997. URL <https://courses.cs.washington.edu/courses/cse590p/06sp/multi.pdf>.
- Jeffrey R Lewis, John Launchbury, Erik Meijer, and Mark B Shields. Implicit parameters: Dynamic scoping with static types. In *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 108–118, 2000. URL <https://doi.org/10.1145/325694.325708>.
- Martin Sulzmann, Gregory J Duck, Simon Peyton-Jones, and Peter J Stuckey. Understanding functional dependencies via constraint handling rules. *Journal of functional programming*, 17(1): 83–129, 2007. URL <https://doi.org/10.1017/S0956796806006137>.

Oleg Kiselyov and Chung-chieh Shan. Functional pearl: implicit configurations—or, type classes reflect the values of types. In *Proceedings of the 2004 ACM SIGPLAN workshop on Haskell*, pages 33–44, 2004. URL [https://d1wqtxts1xzle7.cloudfront.net/43582096/Functional\\_pearl\\_implicit\\_configurations20160310-32037-1bu6179-libre.pdf](https://d1wqtxts1xzle7.cloudfront.net/43582096/Functional_pearl_implicit_configurations20160310-32037-1bu6179-libre.pdf).